Questions

Q1: pthreads中的条件变量（condition variable）有什么用途?

条件变量是一种线程同步机制，用于在多个线程之间高效地等待某个条件的成立。

让线程在某个条件不满足时主动阻塞，并在条件满足时被唤醒继续执行。

Q2: 为何pthreads中的条件变量（condition variable）总是与一个mutex相关联?

- **线程在等待某个条件成立时，需要先检查条件是否满足，但检查条件本身必须是原子操作**
- **通过互斥锁包裹条件的检查和修改，确保“检查条件”和“进入等待”是原子的，避免竞争条件**

Q2: 为何pthreads中的条件变量（condition variable）总是与一个mutex相关联?

条件变量的等待操作 pthread_cond_wait() 实际上会执行以下三步：

- 释放关联的 Mutex（让其他线程可以修改共享数据）。
- 阻塞线程，等待条件变量的信号。
- 被唤醒后，重新获取 Mutex（继续操作共享数据）。

如果没有Mutex：线程无法安全地释放和重新获取锁，可能导致共享数据被破坏。

Q3: 如何自己用pthreads实现一个Barrier（栅栏）？

有哪些方法?

方法1：通过busy-waiting + 互斥锁来实现

方法2：通过使用信号量（Semaphore）来实现

方法3：通过使用互斥锁 + 条件变量来实现

- 使用一个计数器记录已到达的线程数。
- 用互斥锁保护计数器。
- 当计数器未达到总线程数时，线程通过条件变量等待。
- 当最后一个线程到达时，唤醒所有线程。

Implementing a barrier with condition variables

```
/* Shared */
int counter = 0;
pthread_mutex_t mutex;
pthread_cond_t cond_var;
. . .
void* Thread_work(. . .) {
    . . .
    /* Barrier */
    pthread_mutex_lock(&mutex);
    counter++;
    if (counter == thread_count) {
        counter = 0;
        pthread_cond_broadcast(&cond_var);
    } else {
        while (pthread_cond_wait(&cond_var, &mutex) != 0);
    }
    pthread_mutex_unlock(&mutex);
    . . .
}
```

pthread_mutex_unlock(&mutex);
wait_on_signal(&cond_var);
pthread_mutex_lock(&mutex)



Q4: 多个线程同时并行操作一个链表（如执行member、delete、insert操作）会出现什么问题?

会引发一系列线程安全问题，主要包括竞争条件（Race Condition）、数据不一致性、甚至程序崩溃

- **插入（Insert） + 删除（Delete）竞争：**
线程A准备在节点 X 后插入新节点 Y，但线程B同时删除了 X，导致 Y 插入到无效位置或链表断裂。

- **删除（Delete） + 查询（Member）竞争：**
线程A正在读取节点 X 的数据，线程B同时删除 X，导致线程A访问已释放的内存（悬垂指针）